Linux Basics

Grundlagenpraktikum zur IT-Sicherheit (WiSe 2023/2024)

Lehrstuhl für Netz- und Datensicherheit
Ruhr-Universität Bochum

Nico S**** Sabri Öztürk

1. Mai 2026

Zusammenfassung

Dieser Versuch beschäftigt sich mit den Grundlagen von Linux. Linux ist ein weit verbreitetes Betriebssystem, das in vielen Anwendungsbereichen eingesetzt wird. Doch wie wird in diesem System navigiert? Wie werden Dateien gefunden, bewegt und im Netzwerk übertragen? Diese Fragen und viel mehr werden in diesem Protokoll behandelt.

Inhaltsverzeichnis

1 Einleitung **1**

1.1 Glossar 1

2 Aufbau **2**

3 Durchführung **3**

3.1 Navigation im Dateisystem 3

3.2 Dateien Bewegen 4

3.3 Finden von Dateien 5

3.4 Pipelining 6

3.5 Ein praktischer Anwendungsfall 8

3.6 Daten im Netzwerk übertragen 9

3.7 Never have I ever 10

4 Responsible disclosure **11**

5 Zusammenfassung und Fazit **12**

6 LLM **13**

1 Einleitung

Linux ist ein weit verbreitetes Open-Source-Betriebssystem, welches sich mit der Zeit weltweit verbreitet hat. Der Linux-Kernel, welcher von Linus Torvalds, im Jahr 1991 entwickelt wurde, bildet das Herzstück von Linux-basierten Betriebssystemen und ist verantwortlich, für die Verwaltung von Ressourcen, die Kommunikation zwischen Hardware und Software, sowie die Ausführung von Programmen. Linux bietet nicht nur Sicherheit und Stabilität, sondern auch die Möglichkeit, das Betriebsystem auf die benötigten technischen Anforderungen anzupassen und zu konfigurieren. Diese Eigenschaft ist von entscheidender Bedeutung in der heutigen IT-Branche. Linux ermöglicht jeden Einsatzzweck zu optimieren. Egal, ob es für alltägliche Computeraufgaben, für wichtige Geschäftsaufgaben auf einem Server oder für spezielle Aufgaben in kleinen Geräten verwendet werden soll. Linux kann so verändert werden, dass es genau das tut, was gefordert ist. Es bietet die Freiheit, an die jeweiligen Bedürfnissen angepasst zu weden. Zudem ermöglicht die Ressourceneffizienz von Linux eine gute Leistung, selbst auf Hardware mit begrenzten Ressourcen. Die Vielfalt an Open-Source-Software für Linux ermöglicht wiederum, Anwendungen und Werkzeuge zu finden, welche den gewünschten Anforderungen genau entsprechen. In "Linux Basics"wird ein Einblick erlangt, über die wichtigstens Befehele von Linux, das Bewegen und finden von Dateien und Verzeichnissen, was Pipelining ist und wie Datenüber das Netzwerk ausgetauscht werden können. [1]

1.1 Glossar

Hexdump ist die Hexadezimale Darstellung von Daten

Bash ist eine Shell, welche eine Mensch-Maschienen-Schnittstelle bietet

man-page ist eine Seite in der Shell um Informationen über einen Befehl zu erlangen

Flag ist ein weg das Verhalten eines Befehls in der Shell anzupassen

case sensitive Groß- und Kleinbuchstaben beachten

pwd (print working directory) gibt das aktuell aktive Verzeichnis aus

cd (change directory) ändert das aktuell aktive Verzeichnis auf ein angegebenes Verzeichnis

ls (list) gibt eine Liste der im angegebenen Verzeichnis vorhandenen Dateien aus

mv (move) bewegt Datein zwischen Verzeichnissen oder benennt Dateien um

cat schreibt Dateien in die Standardausgabe

2 Aufbau

Die folgenden Versuche wurden im Netzlabor ID 2/168 unternommen.
Wir führen den Versuch zu zweit als Gruppe 43 am PC mit der IP-Adresse 192.168.1.42 durch.
Das Betriebsystem ist ein Ubuntu 22.04 LTS.

Es wurden verschiedene Programme und Befehle genutz. Diese sind:

Software	Version
pwd	GNU coreutils 8.32
cd	GNU coreutils 8.32
ls	GNU coreutils 8.32
mv	GNU coreutils 8.32
grep	GNU grep 3.7
find	GNU findutils 4.8.0
cat	GNU coreutils 8.32
xxd	2021-10-22
tr	GNU coreutils 8.32
echo	GNU coreutils 8.32
sum.py	keine
is_even.py	keine
NetCat	1.10

Einige Versuche werden mit Gruppe 42 am PC mit der IP-Adresse 192.168.1.41 durchgeführt.

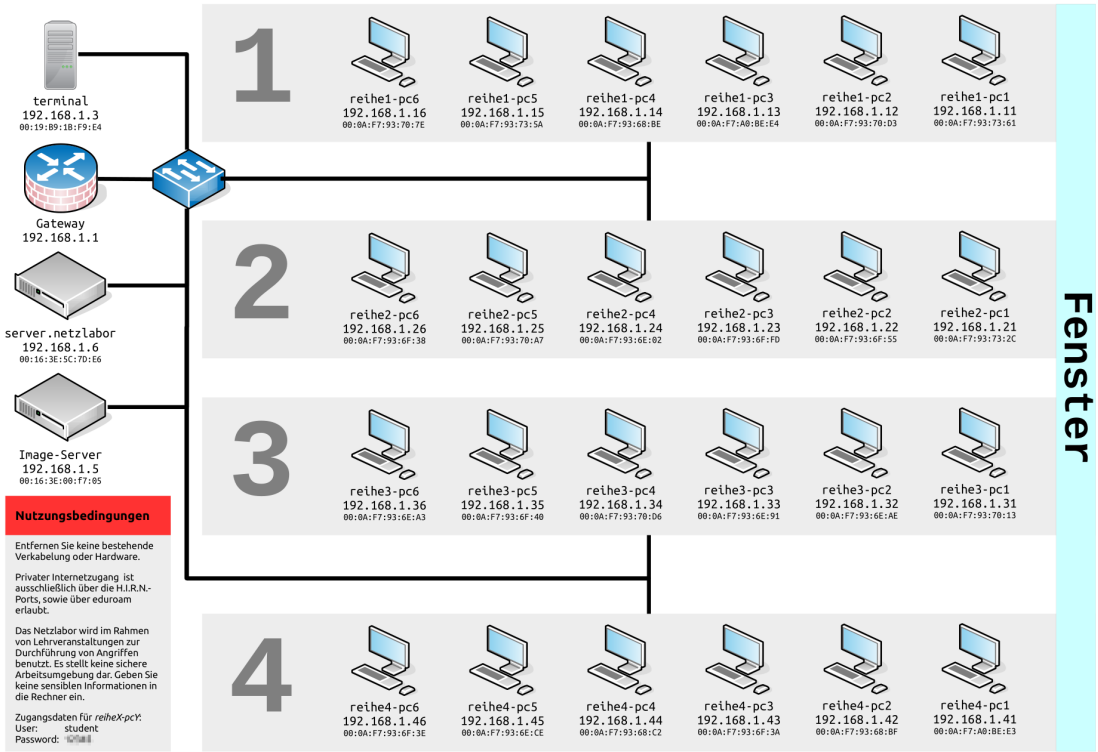

Abbildung 1: Netzplan Raum ID 2/168, Stand 01.05.2017

Die Aufgabenstellung [5] wurde bereitgestellt über die Veranstaltungsseite [4]

3 Durchführung

Im Folgenden werden die einzelnen Aufgaben durchgeführt, erläutert und das Ergebnis präsentiert.

3.1 Navigation im Dateisystem

Im Linux Terminal kann, mit zwei Befehelen, navigiert werden.

Der Befehel *pwd* gibt das aktuelle Verzeichnis aus.

```
student@Reihe4-PC2:~$ pwd
/home/student
```

Um das aktuelle Verzeichnis zu wechseln kann der Befehl *cd*, gefolgt von dem gewünschten Verzeichnis, genutzt werden. So bezieht sich der relative Ausdruck *..* auf das Eltern-Verzeichnis.

```
student@Reihe4-PC2:~$ cd ..
student@Reihe4-PC2:/home$ pwd
/home
```

Um den Inhalt des aktuellen Verzeichnis' auszugeben kann der Befehl *ls* genutzt werden. Wenn der Inhalt eines anderen Verzeichnis' ausgegeben werden soll, kann der Pfad zum gewünschten Verzeichnis als Parameter angegeben werden. Im folgenden ist zu sehen, dass lediglich ein Verzeichnis in */home* existiert, in welchem wir uns vorher befunden haben.

```
student@Reihe4-PC2:/home$ ls
student
```

Durch absolute Pfadangaben kann in jedes beliebige Verzeichnis gewechselt werden. Durch *ls* kann auch hier wieder der Inhalt von */etc/ssh* angezeigt werden.

```
student@Reihe4-PC2:/home$ cd /etc/ssh
student@Reihe4-PC2:/etc/ssh$ ls
moduli        sshd_config         ssh_host_ecdsa_key.pub    ssh_host_rsa_key
ssh_config    sshd_config.d       ssh_host_ed25519_key      ssh_host_rsa_key.pub
ssh_config.d  ssh_host_ecdsa_key  ssh_host_ed25519_key.pub  ssh_import_id
```

Es ist möglich, durch einen einzigen Befehl, mit Hilfe von relativer Pfadangabe, in jedes beliebige Verzeichnis zu wechseln. So kann durch *..* solange in das Eltern-Verzeichnis gewechselt werden, bis das Wurzelverzeichnis */* erreicht ist. Nun kann angegeben werden, in welches Verzeichnis gewechselt werden soll.

```
student@Reihe4-PC2:/etc/ssh$ cd ../../home/student/Praktikum/linux_basics/
student@Reihe4-PC2:~/Praktikum/linux_basics$ pwd
/home/student/Praktikum/linux_basics/
```

3.2 Dateien Bewegen

Um Dateien von einem Verzeichnis in ein anderes zu bewegen, kann der Befehl *mv* genutzt werden. Der erste Parameter nach dem Befehl ist die Quelldatei, welche verschoben werden soll. Der zweite Parameter ist das Zielverzeichnis. Zudem gibt es in Bash den Asterisk-Operator "*", welcher Dateinamen erweitert.

```
student@Reihe4-PC2:~/Praktikum/linux_basics/task_4$ mv unsorted/*kuchen sorted/kuchen/
student@Reihe4-PC2:~/Praktikum/linux_basics/task_4$ mv unsorted/*torte sorted/torte/
student@Reihe4-PC2:~/Praktikum/linux_basics/task_4$ mv unsorted/* sorted/sonstiges/
student@Reihe4-PC2:~/Praktikum/linux_basics/task_4$ ls sorted/kuchen/
apfelkuchen  pflaumenkuchen  streuselkuchen
student@Reihe4-PC2:~/Praktikum/linux_basics/task_4$ ls sorted/torte/
kirschtorte  pflaumentorte  sachertorte
student@Reihe4-PC2:~/Praktikum/linux_basics/task_4$ ls sorted/sonstiges/
brokkoli  mandelpudding  waffeln
```

Es ist ebenfalls möglich, mit dem Befehl *mv*, Dateien umzubenennen. Dazu kann der zweiten Parameter genutzt werden, welcher der neue Dateiname wird.

```
student@Reihe4-PC2:~/Praktikum/linux_basics/task_4$ mv sorted/ sortiert
student@Reihe4-PC2:~/Praktikum/linux_basics/task_4$ ls
sortiert  unsorted
```

Mit *mv* können weiterhin ganze Verzeichnisse verschoben werden.

```
student@Reihe4-PC2:~/Praktikum/linux_basics/task_4$ mv sortiert/ /tmp
```

Der Asterisk-Operator kann zudem genutzt werden, um auf mehrere Verzeichnisse gleichzeitig zu verweisen. So können viele Verzeichnisinhalte gleichzeit mit *ls* ausgegeben werden.

```
student@Reihe4-PC2:~/Praktikum/linux_basics/task_4$ ls -la /tmp/sortiert/*
/tmp/sortiert/kuchen:
insgesamt 8
drwxrwxr-x 2 student student 4096 Okt 18 10:58 .
drwxr-xr-x 5 student student 4096 Okt 18 10:57 ..
-rw-r--r-- 1 student student    0 Mar  9  2021 apfelkuchen
-rw-r--r-- 1 student student    0 Mar  9  2021 pflaumenkuchen
-rw-r--r-- 1 student student    0 Mar  9  2021 streuselkuchen

/tmp/sortiert/sonstiges:
insgesamt 8
drwxrwxr-x 2 student student 4096 Okt 18 10:58 .
drwxr-xr-x 5 student student 4096 Okt 18 10:57 ..
-rw-r--r-- 1 student student    0 Mar  9  2021 brokkoli
-rw-r--r-- 1 student student    0 Mar  9  2021 mandelpudding
-rw-r--r-- 1 student student    0 Mar  9  2021 waffeln

/tmp/sortiert/torte:
insgesamt 8
drwxrwxr-x 2 student student 4096 Okt 18 10:58 .
drwxr-xr-x 5 student student 4096 Okt 18 10:57 ..
-rw-r--r-- 1 student student    0 Mar  9  2021 kirschtorte
-rw-r--r-- 1 student student    0 Mar  9  2021 pflaumentorte
-rw-r--r-- 1 student student    0 Mar  9  2021 sachertorte
```

3.3 Finden von Dateien

Der Befehl *grep* durchsucht den Inhalt aller Dateien im angegebenen Verzeichnis nach einer spezifizierten Zeichenkette.
Durch die Flag *-r* wird rekursiv, in immer tiefer gehenden Verzeichnissen, gesucht.

```
student@Reihe4-PC2:~/Praktikum/linux_basics/task_2$ grep -r 'interblag' ./
./tD/Bs/Ljkt/OfPDmAOq/aq__mRD6o/important:surfin' on the interblag!!!!
./1/VrX1QJnLvX/Xv2468W/KFd3y/hB/auBtViLXt:Dude! Have you seen the interblag???
```

Durch die Flag *-i* ist die Suche nicht mehr case sensitiv.

```
student@Reihe4-PC2:~/Praktikum/linux_basics/task_2$ grep -ir 'world wide blogosphere' ./
./1ZFfQcvA/w/wCAUFi3/x7UWD4ok0/hE2DwK2R:---WoRLD wIDE Blogosphere---
```

Es ist auch möglich, nur nach einzelnen Buchstaben zu suchen.

```
student@Reihe4-PC2:~/Praktikum/linux_basics/task_2$ grep -ir 'é' ./
grep: ./goM1/khRwVtXu/dvYXwh9A/dnRTZPW6Z/PcJ3h1cDgi/U1:  bereinstimmungen  in Binärdatei
./tD/7O:   Qd
              Ouh là là, caché entre des dates binaires! On ne trouvera jamais! Iq5+  "
```

Durch die Flag *-E* können erweiterte reguläre Ausdrücke verwendet werden.

```
student@Reihe4-PC2:~/Praktikum/linux_basics/task_2$ grep -rE '[a-z]+\.[a-z]+@[a-z]+\.[a-
    z]+' ./
./PHL1EgO/zbGPX/PtCH2ulN7/NnQKQGi:Anna: anna.anna@anna.anna
```

Durch den Befehl *find* lassen sich Verzeichnisse durchsuchen. Mit der Flag *-name* lässt sich der Dateiname, der geuschten Datei, spezifizieren.

```
student@Reihe4-PC2:~/Praktikum/linux_basics/task_2$ find . -name 'nobody_will_find_this'
./1ZFfQcvA/MsTz/xb1yDa2p/pi/Wax/nobody_will_find_this
```

Mit Hilfe der Flag *-size* lässt sich die gesuchte Dateigröße festlegen.

```
student@Reihe4-PC2:~/Praktikum/linux_basics/task_2$ find . -name 'important' -size 21c
./goM1/important
```

3.4 Pipelining

Mit Pipelining in Linux können komplexe Aufgaben erledigt werden, indem die Ausgabe eines Befehls direkt in den nächsten geleitet wird. Dies kann Zeit und Ressourcen sparen.

Mit Hilfe des Befehls *cat* kann der Dateiinhalt in der Konsole ausgegeben oder in den Standardinput geschrieben werden.

```
student@Reihe4-PC2:~/Praktikum/linux_basics/task_3$ cat numbers
3 0 7 2 2 1 2
9 6 1 9 5 8 5
3 3 8 8 7 7 9
2 8 7 3 1 8 8
7 1 6 3 1 9 3
9 4 1 2 3 2 4
3 3 0 8 8 1 7
4 3 2 9 0 6 9
3 10 2 6 2 4 7
8 5 5 9 0 1 3
```

Das Programm *sum.py* nimmt zeilenweise Zahlen entgegen und summiert diese auf. Im Anschluss wird die Summe ausgegeben.

```
student@Reihe4-PC2:~/Praktikum/linux_basics/task_3$ ./sum.py
0 1 1
2
2 3 5
10
8 13 21
42
```

Das Programm *is_even.py* nimmt zeilenweise einzelne Zahlen entgegen, um jeweils zu entscheiden, ob die Zahl gerade oder ungerade ist.

```
student@Reihe4-PC2:~/Praktikum/linux_basics/task_3$ ./is_even.py
1
1 is odd
2
2 is even
3
3 is odd
```

Mit Hilfe von *cat* und dem Pipeline-Operator "kann der Inhalt der Datei *numbers* als standard Input an *sum.py* übergeben werden, um die gegebenen Zahlen in *numbers* aufzuaddieren.

```
student@Reihe4-PC2:~/Praktikum/linux_basics/task_3$ cat numbers | ./sum.py
17
43
45
37
30
25
30
33
34
31
```

Durch wiederholtes Pipelining können die von *sum.py* addierten Zahlen an *is_even.py* weitergegeben werden, um zu schauen, welche der Summen gerade oder ungerade ist. Mehrstufige Pipelines werden hierbei von Links nach Rechts ausgeführt. Es werden also erst die Zahlen aus *numbers* an *sum.py* übergeben. Die Ausgabe von *sum.py* wird wiederum an *is_even.py* gegeben.

```
student@Reihe4-PC2:~/Praktikum/linux_basics/task_3$ cat numbers | ./sum.py | ./is_even.
    py
17 is odd
43 is odd
45 is odd
37 is odd
30 is even
25 is odd
30 is even
33 is odd
34 is even
31 is odd
```

Durch die Kombination mit dem *grep* Befehl können die ungeraden Zahlen ausgeben werden.

```
student@Reihe4-PC2:~/Praktikum/linux_basics/task_3$ cat numbers | ./sum.py | ./is_even.
    py | grep 'odd'
17 is odd
43 is odd
45 is odd
37 is odd
25 is odd
33 is odd
31 is odd
```

Die Pipeline kann durch den "<" (kleinerals) Parameter geändert und in einigen Fällen vereinfacht werden.

```
student@Reihe4-PC2:~/Praktikum/linux_basics/task_3$ ./sum.py < numbers | ./is_even.py |
    grep 'odd'
17 is odd
43 is odd
45 is odd
37 is odd
25 is odd
33 is odd
31 is odd
```

3.5 Ein praktischer Anwendungsfall

Um eine zufällige Folge von Bits zu bekommen, kann die Datei *urandom*, in Kombination mit einigen Befehelen, genutzt werden. Der Befehl *head* kann den ersten Teil einer Eingabe ausgeben. Über die Flag *-c* kann angegeben werden, wie viele Byte der Eingabe ausgegeben werden sollen. Wenn also eine Ausgabe von 16 Zeichen gewünscht ist, müssen 8 Byte ausgeben werden, da ein Byte aus zwei Zeichen besteht. Die Datei welche als Eingabe dient */dev/urandom* enthält eine nicht endende Menge an zufälligen Binärdaten. So können, durch das Kombinieren von head und der Datei */dev/urandom*, zufällige Binärdaten der gewünschten Länge erstellt werden. Der Befehl *xxd* kann Binärdaten in hexadezimaler Form ausgeben. Dies ist nützlich, da anderen Falls zufällige Zeichen entstehen und kein, von Menschen lesbares, Hexadezimaltoken. Im *plain text mode*, erkennbar durch die Flag *-p*, werden die Binärdaten als ein zusammenhängender Hexadezimalstring ausgegeben. Um das Token später nutzen zu können, wird es in die Datei */tmp/token* geschrieben.

```
student@Reihe4-PC2:~$ head -c 8 /dev/urandom | xxd -p > /tmp/token
student@Reihe4-PC2:~$ cat /tmp/token
913497201bb59094
```

3.6 Daten im Netzwerk übertragen

Linux ermöglicht das Übertragen von Daten im Netzwerk, wodurch Dateien und Informationen problemlos zwischen verschiedenen Systemen ausgetauscht werden können.

Die Zusammenarbeit erfolgte mit Gruppe 42.
Im Folgenden soll die URL *theuselessweb.com*, über *NetCat*, an den PC von Gruppe 42 mit der IP-Adresse *192.168.1.41* gesendet werden. Zudem wird eine von Gruppe 42 gesendete URL von uns empfangen. Dazu wird *NetCat* im server Modus, über die Flag *-l* (listen) gestartet. *NetCat* ist ein Tool, um Daten über Netzwerkverbindungen zu transportieren. Dazu kann *NetCat* im client oder server Modus gestartet werden. Im client Modus können Daten an einen anderen *NetCat* Prozess im server Modus gesendet werden.

```
student@Reihe4-PC2:~$ echo theuselessweb.com | nc 192.168.1.41 7777
^C
student@Reihe4-PC2:~$ nc -lp 7777
higherlowergame.com
```

Im fogenden soll das Hex-Token *913497201bb59094* über *NetCat* versendet werden. Dazu wird das zuvor erstellte Token aus der Datei */tmp/token* gelesen und an *NetCat* übergeben. Folgend wird das Token der Gruppe 42 empfagen und in der Datei */tmp/neighbor_token* gespeichert.

```
student@Reihe4-PC2:~$ nc 192.168.1.41 7777 < /tmp/token

student@Reihe4-PC2:~$ nc -lp 7777 > /tmp/neighbor_token
^C
student@Reihe4-PC2:~$ cat /tmp/neighbor_token
e837bd8aacea4bea
```

3.7 Never have I ever ...

Der Befehl *rm* wird benutz um Dateien und Verzeichnisse zu löschen. Als ersten Parameter nach dem Befehl wird der Pfad angeben, welcher gelöscht werden soll. Dies birgt jedoch einige Gefahren. Aus der man-page kann abgelesen werden, dass die Flag *-r* rekursiv immer tiefergehende Verzeichnisse ebenfalls löscht. Die Flag *-f* sorgt dafür, dass der Befehl nicht zusätzlich fragt, ob er die gewählte Datei wirklich löschen soll, sondern tut dies einfach. Es kann also schnell der Überblick verloren werden, welche Daten gerade gelöscht werden.
Der Befehl

```
user@host:~$ rm -rf /
```

sorgt also dafür, dass ohne zu fragen, alle Daten ausgehend vom Wurzelverzeichnis rekursiv gelöscht werden. Somit wird das gesamte Dateisystem entfernt und der Computer unbrauchbar gemacht.

Ganz ähnlich funktionert der Befehl

```
user@host:~$ rm -f *
```

Jedoch werden hier lediglich alle Dateien im aktuellen Verzeichnis ohne zu fragen gelöscht. Es werden jedoch auch nicht weitere Verzeichnisse rekursiv durchsucht oder generell Verzeichnisse gelöscht, da die Flag *-r* fehlt. Ebenfalls werden keine verstecken Dateien gefunden. Trotzdem kann auch dieser Befehl erheblichen Schaden anrichten.

Die Zeichenfolge

```
:(){ :|: & };:
```

ist eine sogenannte Bash *Fork Bomb*. Zusammenfassend kann gesagt werden, dass sie den Computer zum abstürzen bringt. Der Anfang `:()` definiert eine Funktion mit dem Namen `:`. Die geschweiften Klammern geben den Anfang und das Ende der Funktion an. `:|:` ruft die Funktion gleich zweimal auf, indem die Ausgabe des rechten Aufrufes in den linken Aufruft geleitet wird. So werden also zwei Kopien der *Fork Bomb* erstellt. Das `&` sorgt dafür, dass der Prozess in den Hintergrund geschoben wird um Systemressourcen zu verbrauchen. Das `;` beendet die Funktionsdefinition und zuletz wird die Funktion mit `:` aufgerufen um den ganzen Prozess zu starten. Es werden nun immer mehr Aufrufe erstellt, die wiederum mehr Aufrufe erstellen, bis das System überlastet ist und abstürzt. [2]

Der Befehl

```
user@host:~$ dd if=/dev/zero of=/dev/sda
```

kann sehr viel Schaden verursachen. Der Befehl *dd* wird grundsätzlich dafür genutz, um Festplatten, Partitionen oder Dateien Bit für Bit zu kopieren. So kann von einer Festplatte schnell und einfach ein Backup auf einer anderen Festplatte gemacht werden. In diesem Beispiel, würde jedoch die gesamte Festplattenpartition */dev/sda* mit Nullen überschrieben. Der Parameter *if* steht für *Input File* und der Parameter *of* für *Output File*. In */dev/zero* sind nur Nullen gespeichert. Die Eingabe sind also lauter Nullen, welche auf */dev/sda* geschrieben werden. Das vernichtet alle Dateien auf dieser Partition. So können in multi-user Systemen auch Partitionen gelöscht werden, die nicht demjenigen gehören, welcher den Befehl ausgeführt hat. Der Befehl kann jedoch auch konstruktiv genutzt werden, wenn eine Festplatte absichtlich formatiert werden soll, also alle Daten überschrieben werden sollen.[3]

4 Responsible disclosure

Sehr geehrte Damen und Herren,

bei einer kürzlich erfolgten Analyse und Überprüfung Ihrer Linux-Systeme wurde ein Fehler in Ihrer Konfiguration gefunden, welcher eine ernstzunehmende Sicherheitslücke darstellt. Es ist möglich, ohne Autorisierung, Zugriff auf die Shell der Linux-Systeme zu erlangen.
Dadurch ist es für Angreifer unter anderem möglich, die Daten auf den Systemen zu kopieren oder zu löschen.

Weiterhin können auch *Denial of Service (DoS)* Angriffe ausgeführt werden, welche die betroffenen Systeme zum Abstürzen bringen können.

Die durch solche Angriffe entstandenen Schäden, sind meist nur mit sehr viel Aufwand wieder zu beheben. Es müssen möglicherweise Systeme neu installiert und Backups übertragen werden. Dies bedeutet einen erheblichen Zeitaufwand.

Um das Problem in der Konfiguration zu beheben, wird der Kontakt zu Ihren Technikern und Systemadministratoren gesucht. Eine geeignete Lösung ist, den Zugang zur Shell besser zu kontrollieren. Durch die Vergabe von Nutzerrechten wird nur bestimmten Personen der Zugang zur Shell gestattet. Diese Lösung wird nach Einschätzugen werder lange dauern, noch aufwendig werden.

Durch die möglicherweise verherenden Ausmaße eines Angriffs, ist die Behebung der System-Lücke schnellstmöglich durchzuführen. Nach 21 Tagen sollte die Lücke geschlossen sein, um Angriffe zu verhindern. Ohne eine Reaktion Ihrerseits wird die Sicherheitslücke der Linux-Systeme nach ablauf der Frist veröffentlicht.

Mit freundlichen Grüßen,

Nico Steppeler und Sabri Öztürk, Berater für IT-Sicherheit

5 Zusammenfassung und Fazit

Dieser Versuch beschäftigte sich mit den Grundlagen von Linux. Durch dieses Protokoll wurde es Benutzern ermöglicht, Dateien und Verzeichnisse in Linux zu suchen, zu finden, zu sortieren, zu verschieben, zu kopieren, umzubenennen und über das Netzwerk zu versenden und zu empfangen. Darüber hinaus wurden Pipelining und einige potentiell problematische Befehle behandelt. Die verschiedenen Befehle und Programme wurden ausgeführt und die jeweiligen Ergebnisse veranschaulicht. Der Einsatz von Linux erstreckt sich weit über die bloße Dateiverwaltung hinaus. Dieses Betriebssystem bietet eine breite Palette von Anwendungen und Möglichkeiten in verschiedenen Bereichen, wie Webservern und Datenbanken.

Gegeben waren miteinander verbundene Copmuter mit einem Linux Betriebsystem. Auf dessen Konsole Befehle ausgeführt wurden. In der Durchführung nutzten wir diese Konsole, um zunächst die Navigation im Dateisystem kennenzulernen. Dann versuchten wir, Dateien in einem Verzeichnis zu bewegen, indem wir sie zuerst nach bestimmten Kriterien in verschiedenen Ordnern aufteilten und für einzelne Dateien unterschiedliche Operationen durchführten. Weiterhin versuchten wir, in verschiedene Arten und Weisen die Dateien zu suchen bzw. zu finden. Durch Pipelining stellten wir unterschiedliche Befehle zusammen und führten sie zusammen aus. Schließlich erstellten wird ein zufälliges Hexadezimaltoken, welches in einer Datei gespeichert wurde. Anschließend tauschten wir mit einer Gruppe jeweils eine URL und dann die generierten Tokens aus. Als Letztes behandelten wir einige Befehle, deren Ausführung mögliche Gefahren mit sich brachte. Dabei wurde auf ihre Nutzung und den Grund ihrer möglichen Gefahren eingegangen. Die Anwendungsmöglichkeiten von Linux sind nicht auf die rein technische Ebene beschränkt. Es ist ein Open-Source-Betriebssystem, das von einer lebendigen Gemeinschaft von Entwicklern und Anwendern unterstützt wird. Dies bedeutet, dass ständige Weiterentwicklung und Innovation an der Tagesordnung sind. Von der Anpassung der Benutzeroberfläche bis hin zur Entwicklung von neuen Anwendungen und Diensten, die auf Linux laufen, bietet die Linux-Gemeinschaft unbegrenzte Möglichkeiten zur Mitgestaltung und Verbesserung.

Die entscheidende Feststellung des Versuchs ist, dass Linux sehr vielfältig einsetzbar ist. Seine umfassende Expertise macht es lohnenswert, Linux auszuprobieren. Dieser Grad an Kontrolle und Anpassungsfähigkeit ist in der Computerwelt selten zu finden und macht Linux zu einer herausragenden Wahl.

6 LLM

In diesem Versuch wurden keine Large-Language Models(LLM) verwendet. Die Texte sind alleine die Arbeit der Mitglieder der Gruppe 43 wie zubeginn angegeben.

Literatur

[1] Bnottelm. Linux. URL https://de.wikipedia.org/wiki/Linux.

[2] Vivek Gite. Understanding bash fork() bomb :() :—: ;: code. URL https://www.cyberciti.biz/faq/understanding-bash-fork-bomb/.

[3] kB. dd. URL https://wiki.ubuntuusers.de/dd/.

[4] Ruhr-Universität Bochum Lehrstuhl für Netz-und Datensicherheit. Bachelor Grundlagenpraktikum ITS: Veranstaltungsseite der Fakultät für Elektrotechnik und Informationstechnik. URL https://informatik.rub.de/studium/lehrveranstaltungen/.

[5] Ruhr-Universität Bochum Lehrstuhl für Netz-und Datensicherheit. Aufgabenstellung zum Praktikumsversuch. Bochum, 2023.